

Lecture 4: Loss Functions

Maximum Likelihood and Cross-Entropy

Firuz Kamalov

MAI 604: Deep Learning

May 14, 2026

The primary goals of today's lecture are to understand:

- 1 The Principle of Maximum Likelihood as a framework for constructing loss functions.
- 2 How to derive standard loss functions for Regression (Least Squares) and Classification (Cross-Entropy).

Contents

- ➊ Maximum Likelihood
Motivating Examples
MLE in Deep Learning
- ➋ Recipe for Loss Functions
- ➌ Univariate Regression
- ➍ Binary Classification
- ➎ Multiclass Classification
- ➏ Cross-Entropy vs NLL
- ➐ Summary

Contents

- ➊ Maximum Likelihood
Motivating Examples
MLE in Deep Learning
- ➋ Recipe for Loss Functions
- ➌ Univariate Regression
- ➍ Binary Classification
- ➎ Multiclass Classification
- ➏ Cross-Entropy vs NLL
- ➐ Summary

Maximum Likelihood Estimation (MLE)

In Maximum Likelihood Estimation, we assume that the observed data follow a probability distribution $P(y \mid \theta)$.

Our goal is to find the parameter values that maximize the likelihood of observing the data y :

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} P(y \mid \theta)$$

Example

If we observe 7 heads in 10 coin flips, then the MLE estimate is:

$$\hat{p} = 0.7$$

because $p = 0.7$ makes the observed data most probable.

Example 1: Coin Flip

Scenario: Suppose we wish to determine the probability of a biased coin. We toss the coin $n = 5$ times.

- **Data:** We observe data: 3 Heads and 2 Tails.
- **Model:** $Pr(\text{Data}|p) = p^3(1 - p)^2$
- **Goal:** Determine the **most likely** value of parameter p .

$$L(p) = p^3(1 - p)^2$$

$$\begin{aligned}\frac{d}{dp}L(p) &= 3p^2(1 - p)^2 - 2p^3(1 - p) \\ &= p^2(1 - p)(3 - 5p)\end{aligned}$$

Setting the derivative equal to zero:

$$\hat{p}_{\text{MLE}} = \frac{3}{5} = 0.6$$

Example: MLE for the Mean (μ)

Scenario: We have I independent observations $\{y_1, y_2, \dots, y_I\}$ from a Normal distribution where σ^2 is known, but μ is unknown.

1. Likelihood Function:

$$L(\mu) = \prod_{i=1}^I \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y_i - \mu)^2}{2\sigma^2} \right]$$

2. Negative Log-Likelihood:

$$L[\mu] = \sum_{i=1}^I \frac{(y_i - \mu)^2}{2\sigma^2}$$

3. Optimization:

$$\frac{\partial L[\mu]}{\partial \mu} = \sum_{i=1}^I \frac{2(y_i - \mu)}{2\sigma^2}(-1) = 0 \implies \sum_{i=1}^I (y_i - \mu) = 0$$

4. Result: $\sum_{i=1}^I y_i = I\mu \implies \hat{\mu} = \frac{1}{I} \sum_{i=1}^I y_i$

Maximum Likelihood in Deep Learning

We are given a dataset of input/output pairs $\{(\mathbf{x}_i, y_i)\}_{i=1}^I$

Our goal is to **maximize the probability** of observing the data $\{y_i\}$ under a probabilistic model:

$$L(\phi) = \prod_{i=1}^I P(y_i \mid \theta_i),$$

where the model parameters are estimated by the neural network

$$\theta_i = \mathbf{f}(\mathbf{x}_i; \phi)$$

The optimal model is obtained by maximizing the likelihood function:

$$\hat{\phi} = \arg \max_{\phi} \prod_{i=1}^I P(y_i \mid \mathbf{f}(\mathbf{x}_i; \phi)).$$

Example: Exponential Likelihood

Suppose the target variable satisfies $y_i \geq 0$ and follows an exponential distribution:

$$P(y_i \mid \lambda_i) = \lambda_i e^{-\lambda_i y_i},$$

where the rate parameter is predicted by the neural network:

$$\lambda_i = f(\mathbf{x}_i; \phi).$$

The likelihood function for the dataset is

$$L(\phi) = \prod_{i=1}^I \lambda_i e^{-\lambda_i y_i}$$

The optimal neural network parameters are obtained by

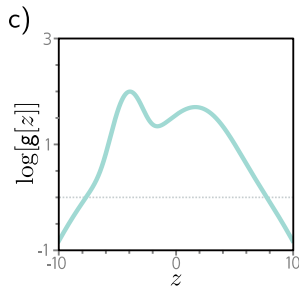
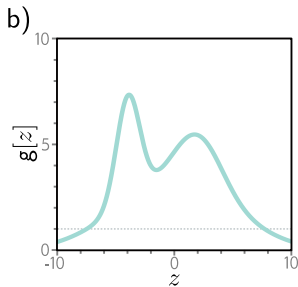
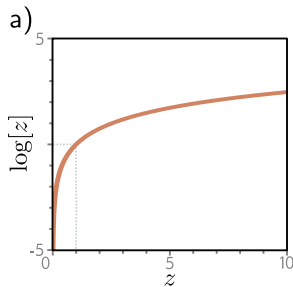
$$\hat{\phi} = \arg \max_{\phi} \prod_{i=1}^I f(\mathbf{x}_i; \phi) \exp[-f(\mathbf{x}_i; \phi) y_i]$$

Maximizing Log-Likelihood

Using the product of probabilities is numerically unstable (values become tiny).

Since the logarithm is monotonically increasing, maximizing the likelihood is equivalent to maximizing the **log-likelihood**:

$$\hat{\phi} = \operatorname{argmax}_{\phi} \sum_{i=1}^I \log [Pr(y_i | \mathbf{f}(\mathbf{x}_i, \phi))] \quad (1)$$



Minimizing Negative Log-Likelihood

By convention, we frame problems as minimizing a loss. We multiply by -1 to get the negative log-likelihood criterion:

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[- \sum_{i=1}^I \log [Pr(y_i | \mathbf{f}[\mathbf{x}_i, \phi])] \right] = \operatorname{argmin}_{\phi} L[\phi] \quad (2)$$

Inference: To predict a value, we typically take the maximum of the predicted distribution:

$$\hat{y} = \operatorname{argmax}_y \left[Pr(y | \mathbf{f}[\mathbf{x}, \hat{\phi}]) \right]$$

Contents

- ➊ Maximum Likelihood
 - Motivating Examples
 - MLE in Deep Learning
- ➋ Recipe for Loss Functions
- ➌ Univariate Regression
- ➍ Binary Classification
- ➎ Multiclass Classification
- ➏ Cross-Entropy vs NLL
- ➐ Summary

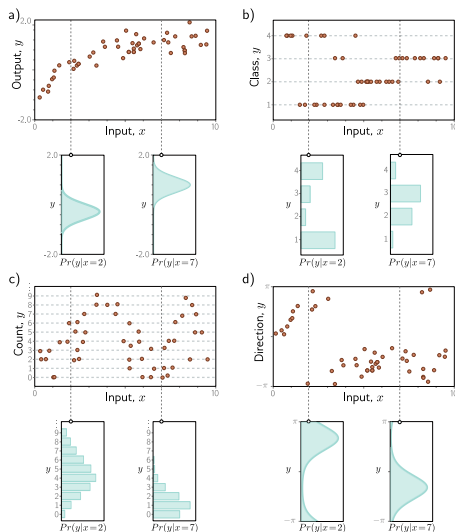
Recipe for Constructing Loss Functions

- 1 **Choose a distribution** $Pr(y|\theta)$ defined over the output domain y .
- 2 **Set the model** $\mathbf{f}[\mathbf{x}, \phi]$ to predict the distribution parameters θ .
- 3 **Train** by minimizing the negative log-likelihood loss:

$$L[\phi] = - \sum_{i=1}^I \log Pr(y_i | \mathbf{f}[\mathbf{x}_i, \phi])$$

- 4 **Inference:** Return the full distribution or the maximum.

Choosing the Right Distribution



Contents

- ➊ Maximum Likelihood
 - Motivating Examples
 - MLE in Deep Learning
- ➋ Recipe for Loss Functions
- ➌ Univariate Regression
- ➍ Binary Classification
- ➎ Multiclass Classification
- ➏ Cross-Entropy vs NLL
- ➐ Summary

Univariate Regression: Problem Setup

Definition: Predict a continuous scalar output $y \in \mathbb{R}$ based on input data $\mathbf{x}$.

Real-world Examples:

(i) House Price Prediction

- **Data $\mathbf{x}$:** Property features (square footage, number of rooms, location).
- **Target y :** The market price (e.g., a value like 350,400).

(ii) Temperature Forecasting

- **Data $\mathbf{x}$:** Atmospheric sensor readings (humidity, pressure, wind speed).
- **Target y :** The temperature at a specific time (e.g., a value like 24.5°C).

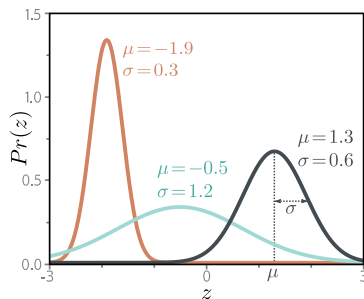
Univariate Regression: The Normal Distribution

Goal: Predict scalar $y \in \mathbb{R}$.

Step 1: Choose Univariate Normal Distribution.

$$Pr(y|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y - \mu)^2}{2\sigma^2} \right]$$

Step 2: Model predicts the mean $\mu = \mathbf{f}[\mathbf{x}, \phi]$. Variance σ^2 is often treated as constant.



Deriving Least Squares Loss

Step 3: Minimize Negative Log-Likelihood:

$$L[\phi] = - \sum_{i=1}^I \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{(y_i - \mathbf{f}[\mathbf{x}_i, \phi])^2}{2\sigma^2} \right] \right]$$

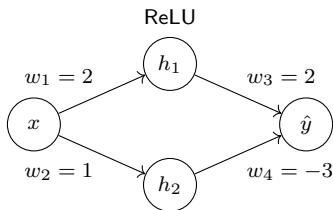
Removing terms that do not depend on ϕ and constant scaling factors:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} \sum_{i=1}^I (y_i - \mathbf{f}[\mathbf{x}_i, \phi])^2$$

Result: The **Least Squares** loss follows naturally from the assumption of Gaussian errors.

Numerical Example: Least Squares Loss

1. Network Architecture



Input

Output

2. Individual Loss

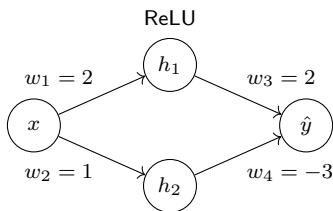
Point 1: $(x_1 = 2, y_1 = 4)$

Model Equations:

- $\phi = [w_1, w_2, w_3, w_4]$
- $\mathbf{f}[x_i, \phi] = \hat{y}_i$
- $L = \sum_{i=1}^N (y_i - \hat{y}_i)^2$

Numerical Example: Least Squares Loss

1. Network Architecture



Input

Output

Model Equations:

- $\phi = [w_1, w_2, w_3, w_4]$
- $\mathbf{f}[x_i, \phi] = \hat{y}_i$
- $L = \sum_{i=1}^N (y_i - \hat{y}_i)^2$

2. Individual Loss

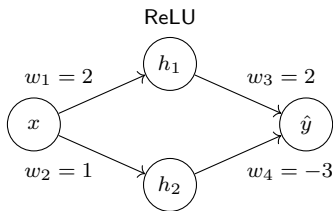
Point 1: $(x_1 = 2, y_1 = 4)$

- $\hat{y}_1 = (4 \cdot 2) + (2 \cdot -3) = 2$
- $Loss_1 = (4 - 2)^2 = 4$

Point 2: $(x_2 = -1, y_2 = -0.2)$

Numerical Example: Least Squares Loss

1. Network Architecture



Input

Output

Model Equations:

- $\phi = [w_1, w_2, w_3, w_4]$
- $\mathbf{f}[x_i, \phi] = \hat{y}_i$
- $L = \sum_{i=1}^N (y_i - \hat{y}_i)^2$

2. Individual Loss

Point 1: $(x_1 = 2, y_1 = 4)$

- $\hat{y}_1 = (4 \cdot 2) + (2 \cdot -3) = 2$
- $Loss_1 = (4 - 2)^2 = 4$

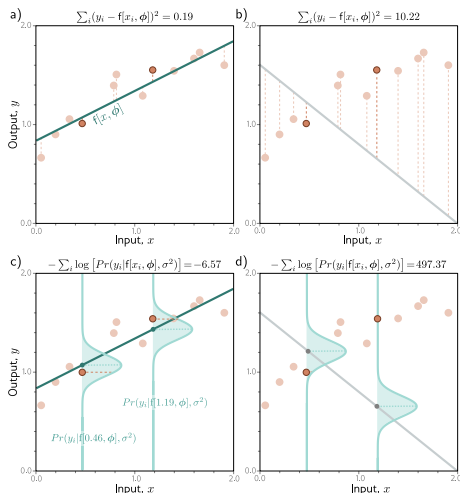
Point 2: $(x_2 = -1, y_2 = -0.2)$

- $\hat{y}_2 = (0 \cdot 2) + (0 \cdot -3) = 0$
- $Loss_2 = (-0.2 - 0)^2 = 0.04$

3. Total Least Squares Loss

$$L = L_1 + L_2 = 4 + 0.04 = 4.04$$

Visualization of Regression Loss

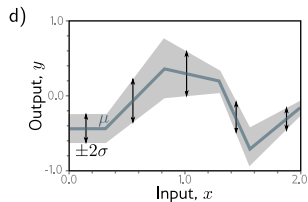
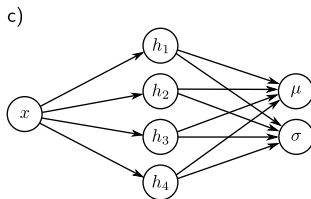
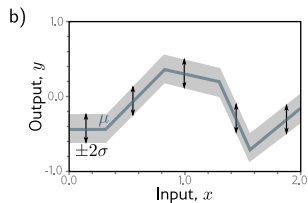
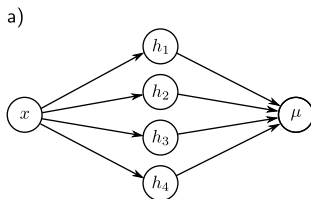


Least squares minimizes squared deviations (top). This is equivalent to maximizing the probability density of the data under the model's predicted Normal distribution (bottom).

Heteroscedastic Regression

We can also predict the uncertainty σ^2 as a function of the input.

- **Homoscedastic:** Variance σ^2 is constant.
- **Heteroscedastic:** Variance depends on input $\sigma^2 = f_2[\mathbf{x}, \phi]^2$.



Contents

- ➊ Maximum Likelihood
 - Motivating Examples
 - MLE in Deep Learning
- ➋ Recipe for Loss Functions
- ➌ Univariate Regression
- ➍ Binary Classification
- ➎ Multiclass Classification
- ➏ Cross-Entropy vs NLL
- ➐ Summary

Binary Classification: Problem Setup

Definition: Assign input data $\mathbf{x}$ to one of two discrete classes $y \in \{0, 1\}$, referred to as the label.

Examples:

(i) Sentiment Analysis

- **Data x :** Text from a restaurant review.
- **Label y :**
 - $y = 1$: Positive
 - $y = 0$: Negative

(ii) Medical Diagnosis

- **Data x :** An MRI scan image.
- **Label y :**
 - $y = 1$: Tumor present
 - $y = 0$: Tumor absent

Binary Classification: Bernoulli Distribution

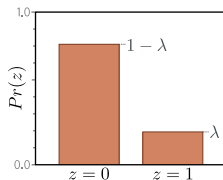
Goal: Predict label $y \in \{0, 1\}$.

Step 1: Choose Bernoulli Distribution.

$$Pr(y|\lambda) = \begin{cases} 1 - \lambda & y = 0 \\ \lambda & y = 1 \end{cases}$$

Or compactly: $Pr(y|\lambda) = (1 - \lambda)^{1-y} \cdot \lambda^y$.

Here $\lambda \in [0, 1]$ is the probability that $y = 1$.

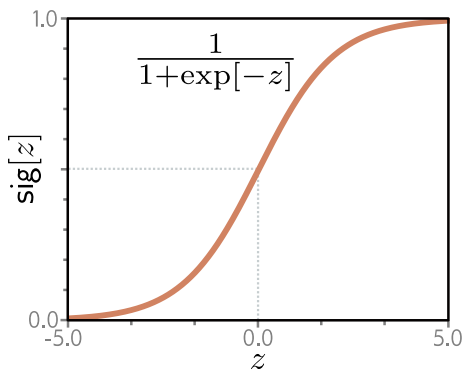


The Logistic Sigmoid

Step 2: Model predicts λ .

Since the network output can be any real number, we map it to $[0, 1]$ using the **logistic sigmoid** function:

$$\text{sig}[z] = \frac{1}{1 + \exp[-z]}$$



Binary Cross-Entropy Loss

Step 3: Minimize Negative Log-Likelihood.

$$L[\phi] = - \sum_{i=1}^I \log [(1 - \lambda_i)^{1-y_i} \cdot \lambda_i^{y_i}]$$

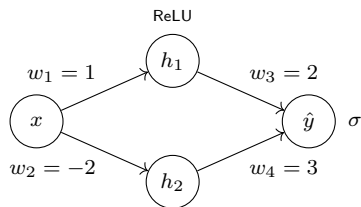
Substituting $\lambda_i = \text{sig}[\mathbf{f}[\mathbf{x}_i, \phi]]$:

$$L[\phi] = - \sum_{i=1}^I [(1 - y_i) \log(1 - \text{sig}[\mathbf{f}[\mathbf{x}_i, \phi]]) + y_i \log(\text{sig}[\mathbf{f}[\mathbf{x}_i, \phi]])] \quad (3)$$

This is known as the **Binary Cross-Entropy** loss.

Numerical Example: Binary Cross-Entropy Loss

1. Network Architecture



2. Individual Loss

Point 1: $(x_1 = 2, y_1 = 1)$

Key Equations:

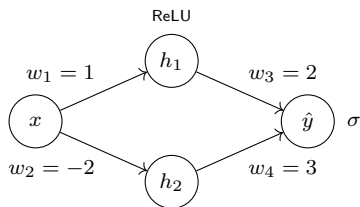
$$L = - \sum_{i=1}^2 [y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)]$$

$$\hat{y}_i = \text{sig}[\mathbf{f}[x_i, \phi]]$$

$$L \rightarrow 0 \text{ as } \hat{y} \rightarrow y$$

Numerical Example: Binary Cross-Entropy Loss

1. Network Architecture



Key Equations:

$$L = - \sum_{i=1}^2 [y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)]$$

$$\hat{y}_i = \text{sig}[\mathbf{f}[x_i, \phi]]$$

$$L \rightarrow 0 \text{ as } \hat{y} \rightarrow y$$

2. Individual Loss

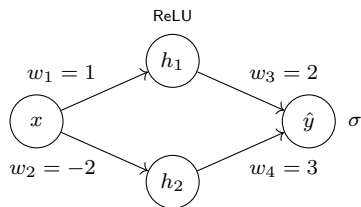
Point 1: $(x_1 = 2, y_1 = 1)$

- $z = (2 \cdot 2) + (0 \cdot 3) = 4$
- $\hat{y}_1 = \sigma(4) \approx \mathbf{0.98}$
- $L_1 = -(1 \cdot \ln(0.98)) \approx \mathbf{0.02}$

Point 2: $(x_2 = 1, y_2 = 0)$

Numerical Example: Binary Cross-Entropy Loss

1. Network Architecture



Key Equations:

$$L = - \sum_{i=1}^2 [y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)]$$

$$\hat{y}_i = \text{sig}[\mathbf{f}[x_i, \phi]]$$

$$L \rightarrow 0 \text{ as } \hat{y} \rightarrow y$$

2. Individual Loss

Point 1: ($x_1 = 2, y_1 = 1$)

- $z = (2 \cdot 2) + (0 \cdot 3) = 4$
- $\hat{y}_1 = \sigma(4) \approx \mathbf{0.98}$
- $L_1 = -(1 \cdot \ln(0.98)) \approx \mathbf{0.02}$

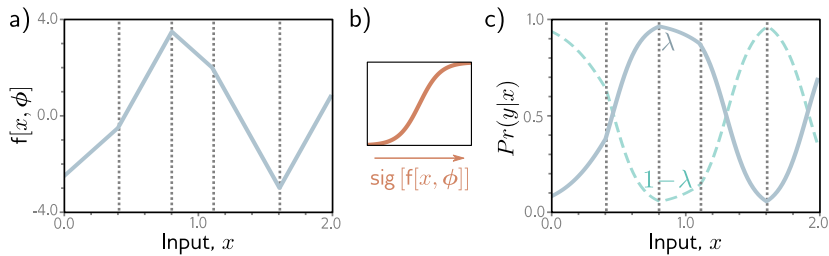
Point 2: ($x_2 = 1, y_2 = 0$)

- $z = (1 \cdot 2) + (0 \cdot 3) = 2$
- $\hat{y}_2 = \sigma(2) \approx \mathbf{0.88}$
- $L_2 = -(1 - 0) \ln(1 - 0.88) = -\ln(0.12) \approx \mathbf{2.12}$

3. Total BCE Loss:

$$L = L_1 + L_2 = 0.02 + 2.12 = \mathbf{2.14}$$

Visualizing Binary Classification



a) Network output (unbounded). b) After Sigmoid (0 to 1). c) Probability of true label. Loss encourages λ to be high when $y = 1$ and low when $y = 0$.

Contents

- ➊ Maximum Likelihood
 - Motivating Examples
 - MLE in Deep Learning
- ➋ Recipe for Loss Functions
- ➌ Univariate Regression
- ➍ Binary Classification
- ➎ Multiclass Classification
- ➏ Cross-Entropy vs NLL
- ➐ Summary

Multiclass Classification: Problem Setup

Definition: Assign an input $\mathbf{x}$ to one of $K > 2$ classes, denoted by the label $y \in \{1, 2, \dots, K\}$.

Real-world Examples:

- Predicting which of $K = 10$ digits y is present in an image $\mathbf{x}$ of a handwritten number
- Predicting which of K possible words y follows an incomplete sentence $\mathbf{x}$

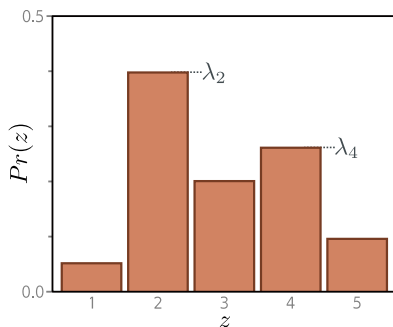
Multiclass Classification: Categorical Distribution

Goal: Predict one of K classes $y \in \{1, \dots, K\}$.

Step 1: Choose Categorical Distribution with parameters $\lambda_1, \dots, \lambda_K$.

$$Pr(y = k) = \lambda_k$$

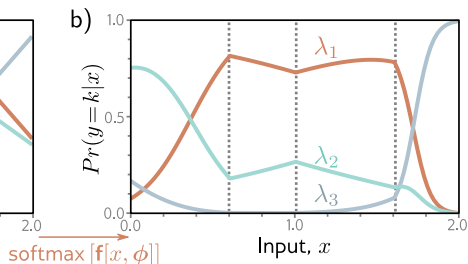
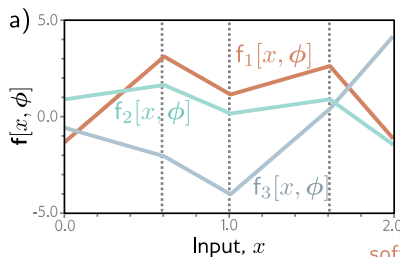
Constraint: $\sum \lambda_k = 1$ and $\lambda_k \geq 0$.



The Softmax Function

Step 2: Model predicts the parameters $[\lambda_1, \lambda_2, \dots, \lambda_k]$. To ensure constraints, we use the **Softmax** function:

$$\text{softmax}_k[\mathbf{z}] = \frac{\exp[z_k]}{\sum_{k'=1}^K \exp[z_{k'}]}$$



Multiclass Cross-Entropy Loss

Step 3: Minimize Negative Log-Likelihood.

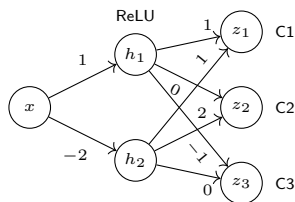
$$L[\phi] = - \sum_{i=1}^I \log (\text{softmax}_{y_i} [\mathbf{f}[\mathbf{x}_i, \phi]])$$

$$L[\phi] = - \sum_{i=1}^I \left(f_{y_i}[\mathbf{x}_i, \phi] - \log \left[\sum_{k'=1}^K \exp(f_{k'}[\mathbf{x}_i, \phi]) \right] \right)$$

This is the **Multiclass Cross-Entropy** loss. It tries to make the model output for the correct class y_i as large as possible relative to the others.

Numerical Example: Multiclass Cross-Entropy

1. Network Architecture (K=3)



2. Individual Loss

Point 1: ($x_1 = 2, y_1 = C_3$)

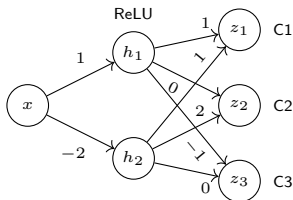
Network Output:

$$\text{softmax}(\mathbf{f}[\mathbf{x}, \phi]) = \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \end{bmatrix}$$

$$\text{softmax}(\mathbf{f}[2, \phi]) = \begin{bmatrix} 87\% \\ 11.5\% \\ 1.5\% \end{bmatrix}$$

Numerical Example: Multiclass Cross-Entropy

1. Network Architecture (K=3)



Network Output:

$$\text{softmax}(\mathbf{f}[\mathbf{x}, \phi]) = \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \end{bmatrix}$$

$$\text{softmax}(\mathbf{f}[2, \phi]) = \begin{bmatrix} 87\% \\ 11.5\% \\ 1.5\% \end{bmatrix}$$

2. Individual Loss

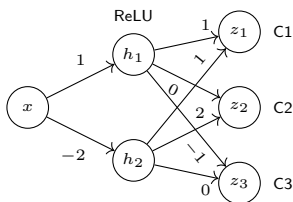
Point 1: ($x_1 = 2, y_1 = C_3$)

- $z_1 = 2(1) + 0(1) = 2$
- $z_2 = 2(0) + 0(2) = 0$
- $z_3 = 2(-1) + 0(0) = -2$
- $P_3 = \frac{e^{-2}}{e^2 + e^0 + e^{-2}} = \frac{0.13}{7.39 + 1 + 0.13} \approx \mathbf{0.0153}$
- $L_1 = -\ln(0.0153) \approx \mathbf{4.1799}$

Point 2: ($x_2 = 1, y_2 = C_2$)

Numerical Example: Multiclass Cross-Entropy

1. Network Architecture (K=3)



Network Output:

$$\text{softmax}(\mathbf{f}[\mathbf{x}, \phi]) = \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \end{bmatrix}$$

$$\text{softmax}(\mathbf{f}[2, \phi]) = \begin{bmatrix} 87\% \\ 11.5\% \\ 1.5\% \end{bmatrix}$$

2. Individual Loss

Point 1: ($x_1 = 2, y_1 = C_3$)

- $z_1 = 2(1) + 0(1) = 2$
- $z_2 = 2(0) + 0(2) = 0$
- $z_3 = 2(-1) + 0(0) = -2$
- $P_3 = \frac{e^{-2}}{e^2 + e^0 + e^{-2}} = \frac{0.13}{7.39 + 1 + 0.13} \approx \mathbf{0.0153}$
- $L_1 = -\ln(0.0153) \approx \mathbf{4.1799}$

Point 2: ($x_2 = 1, y_2 = C_2$)

- $z_1 = 1, z_2 = 0, z_3 = -1$
- $P_2 = \frac{1}{e^1 + e^0 + e^{-1}} \approx \mathbf{0.24}$
- $L_2 = -\ln(0.24) \approx \mathbf{1.43}$

3. Total Cross-Entropy Loss

$$L = L_1 + L_2 = 4.18 + 1.43 = \mathbf{5.61}$$

Simplified Notation: Probabilities vs Logits

In general literature, the output of the network is often assumed to be converted into a probability p :

- **Binary:** $p = \text{sig}[z] \in (0, 1)$
- **Multiclass:** $p_k = \text{softmax}_k[\mathbf{z}] \in (0, 1)$

Simplified Loss Formulas

- **Binary Cross-Entropy:**

$$L = - \sum_{i=1}^I [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

- **Multiclass Cross-Entropy:**

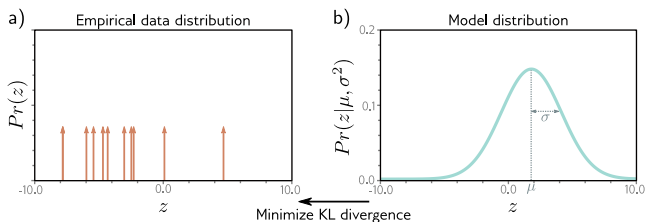
$$L = - \sum_{i=1}^I \sum_{k=1}^K y_{i,k} \log(p_{i,k})$$

Contents

- ➊ Maximum Likelihood
 - Motivating Examples
 - MLE in Deep Learning
- ➋ Recipe for Loss Functions
- ➌ Univariate Regression
- ➍ Binary Classification
- ➎ Multiclass Classification
- ➏ Cross-Entropy vs NLL
- ➐ Summary

What is Cross-Entropy?

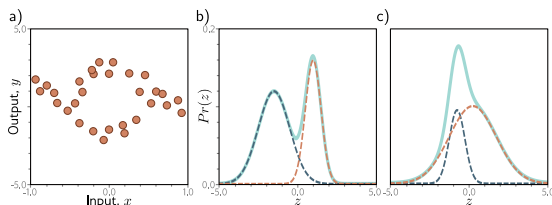
The term "cross-entropy" comes from minimizing the **Kullback-Leibler (KL) Divergence** between the empirical data distribution $q(y)$ and the model distribution $Pr(y|\theta)$.



Key Insight: Minimizing the KL divergence between the empirical distribution (point masses at data) and the model distribution is **mathematically equivalent** to minimizing the Negative Log-Likelihood.

Complex Distributions

The recipe applies to more complex data types too. For example, multimodal data can be modeled using a Mixture of Gaussians (Mixture Density Networks).



Example: Predicting a distribution that has multiple peaks (modes).

Data Type	Domain	Distribution	Use
univariate, continuous, unbounded	$y \in \mathbb{R}$	univariate normal	regression
univariate, continuous, unbounded	$y \in \mathbb{R}$	Laplace or t-distribution	robust regression
univariate, continuous, unbounded	$y \in \mathbb{R}$	mixture of Gaussians	multimodal regression
univariate, continuous, bounded below	$y \in \mathbb{R}^+$	exponential or gamma	predicting magnitude
univariate, continuous, bounded	$y \in [0, 1]$	beta	predicting proportions
multivariate, continuous, unbounded	$\mathbf{y} \in \mathbb{R}^K$	multivariate normal	multivariate regression
univariate, continuous, circular	$y \in (-\pi, \pi]$	von Mises	predicting direction
univariate, discrete, binary	$y \in \{0, 1\}$	Bernoulli	binary classification
univariate, discrete, bounded	$y \in \{1, 2, \dots, K\}$	categorical	multiclass classification
univariate, discrete, bounded below	$y \in \{0, 1, 2, 3, \dots\}$	Poisson	predicting event counts
multivariate, discrete, permutation	$\mathbf{y} \in \text{Perm}[1, 2, \dots, K]$	Plackett-Luce	ranking

Contents

- ➊ Maximum Likelihood
 - Motivating Examples
 - MLE in Deep Learning
- ➋ Recipe for Loss Functions
- ➌ Univariate Regression
- ➍ Binary Classification
- ➎ Multiclass Classification
- ➏ Cross-Entropy vs NLL
- ➐ Summary

Summary

- **Loss Function Design:** We use the Maximum Likelihood principle.
- **Recipe:**
 - ① Pick a distribution $Pr(y|\theta)$.
 - ② Predict θ with a neural network.
 - ③ Minimize Negative Log-Likelihood (NLL).

- **Loss Functions**

- **Regression:**

$$L = \sum_{i=1}^I (y_i - \hat{y}_i)^2$$

- **Binary Cross-Entropy:**

$$L = - \sum_{i=1}^I [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

- **Multiclass Cross-Entropy:**

$$L = - \sum_{i=1}^I \sum_{k=1}^K y_{i,k} \log(\hat{p}_{i,k})$$